
seLIA2: evaluación formativa de código Python con IA local explicable

Rúbrica criterial · SHAP verificado · Feedback pedagógico

Ricardo Hidalgo Aragón

Universidad Rey Juan Carlos

Ruff

Radon

AST

SHAP

Qwen 1.5B

100 % local · sin APIs · rúbrica auditable · feedback en 3 niveles

El reto: evaluar código de estudiantes de forma justa y privada



El problema de la evaluación

- Decenas de entregas por semana
- Privacidad del alumnado (RGPD):
el código es dato personal
- APIs de IA: coste + datos a terceros
- El feedback genérico no enseña:
“revisa las buenas prácticas”
- Las herramientas actuales solo dicen
qué falla, no *cómo* mejorar

La propuesta: seLIA2

Pipeline **100 % local** que combina:

1. Análisis estático (Ruff + Radon + AST)
→ hechos objetivos y reproducibles
2. Rúbrica criterial (0-100, 4 dimensiones)
→ nota auditable con criterios públicos
3. SHAP con garantías de fidelidad
→ explica el *por qué* de la nota
4. Feedback pedagógico en 3 niveles
→ qué falla, cómo corregirlo, qué reflexionar

¿Por qué no simplemente usar una API de IA?



API comercial (OpenAI, Anthropic, ...)

- ~0,20–1,50 € por entrega
- Datos del alumno a terceros
- No reproducible (versión, logging policy)
- “Porque la IA lo dice”
- Internet + API key + dependencia externa

La propuesta (100 % local)

- 0 € por entrega
- Datos **nunca** salen del aula
- Diagnóstico **reproducible** (Ruff, Radon, SHAP)
- Nota con **justificación auditable**
- Python 3.12 + 4 GB RAM

La clave: **diagnóstico reproducible** con rúbrica pública, sin comprometer privacidad

Estación 1 — Análisis estático: 50 features objetivas



Ruff — linter (13 reglas)

24 features: códigos individuales (F401, F841, E501, F811, F821, E711, E712, E722, W291, W293, C901, PLR091), totales, severidades, familias.

--isolated --select: mismas

reglas para todos los estudiantes.

Radon — complejidad

15 features: CC avg/max, LOC, LLOC, SLOC, MI, Halstead (volumen, dificultad, esfuerzo), num_functions, ratios (cc/func, effort/loc, vol/func, logical_density).

AST — estructura

11 features: cobertura de docstrings y type hints, anidamiento máximo, números mágicos, bare excepts, globals, clases, longitud función (avg/max), parámetros largos, nombres cortos.

En **milisegundos**, sin conexión, sin modelos: código fuente → **50 features deterministas y reproducibles**.

Ablaciones disponibles: --no-metric (sin Radon, 35 feat.), --no-ast (sin AST, 39 feat.)

Estación 2 — Rúbrica criterial: nota 0–100 con 4 dimensiones



Dimensión	Peso	Max	Penaliza / Premia
Corrección	40 %	40	Errores pyflakes (F), bare excepts
Estilo	20 %	20	PEP 8 (E/W), nombres cortos
Complejidad	20 %	20	CC alto, anidamiento, MI bajo
Documentación	20 %	20	Docstrings, type hints, comentarios

Calificación oficial RD 1125/2003 (nota/10): SB Sobresaliente ≥ 9 · NT Notable ≥ 7 · AP Aprobado ≥ 5 · SS Suspenso < 5 .

¿Por qué una rúbrica criterial?

- **Sadler (1989)**: el estudiante necesita conocer el estándar para cerrar la brecha
- **Nicol & Macfarlane-Dick (2006)**: criterios explícitos facilitan la autoevaluación
- Cada penalización queda **registrada y justificada**
- El educador puede **auditar** todos los umbrales en config.py

Ejemplo: student_bad.py

Corrección: -16 ($4 \times F401 + 2 \times F841$)

Estilo: -6 (W291, W293)

Complejidad: -4 (CC max = 15)

Documentación: -8 (sin docstrings ni type hints)

Nota: $6/100 = 0,6/10 \rightarrow$ Suspenso (SS)

Nota con justificación auditable y criterios públicos

Ampliación: ¿Qué herramienta aporta cada dimensión?



Corrección (40 %)

Ruff: F821/F823 → -10 c/u; resto F → -8 c/u

AST: bare_excepts → -4 c/u

Estilo (20 %)

Ruff: E/W (PEP~8) → -2 c/u (cap. -20)

AST: short_name >50 % → -3

Complejidad (20 %)

Radon: CC avg >10 → -8; >5 → -4

MI <50 → -6; <65 → -3

AST: nesting >4 → -6; >3 → -3

func >50L → -4; params/global → -2 c/u

Documentación (20 %)

AST: docstrings → +10; type hints → +6

magic nums >3 → -2

Radon: comments/sloc → +4

Todas las penalizaciones quedan en RubricResult.penalties → informe Markdown.

Umbral auditables en selia2/config.py.

Estación 3 — SHAP verificado: fidelidad y estabilidad



Surrogate + diagnósticos de calidad

RandomForest (150 árboles, profundidad 4)
predice aprobado/suspendido (score ≥ 50 , RD 1125/2003).

Fidelidad: CV accuracy del surrogate.

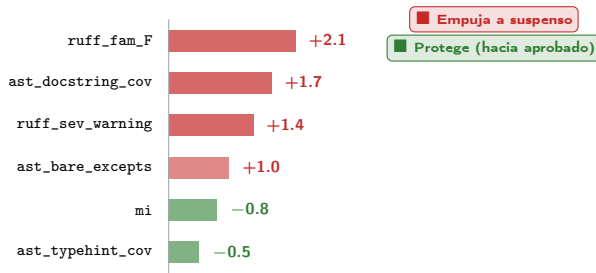
Si $< 0,7 \rightarrow$ **aviso al docente**.

Estabilidad: Spearman del ranking
de |SHAP| entre 3 semillas.

Con 120 archivos sintéticos:

fid. $\approx 0,98$ · estab. $\approx 0,94$

Valores SHAP — student_bad.py (50 feat.)



Las features AST (docstrings, bare excepts, type hints) aparecen junto a Ruff/Radon: la explicación es rica porque el dataset tiene 120+ archivos con perfiles diversos.

Estación 4 — Feedback en 3 niveles: Hattie & Timperley



Marco teórico

Hattie & Timperley (2007): el feedback eficaz responde a tres preguntas y opera en niveles:

Tarea: ¿qué falla? (nivel básico)

Proceso: ¿cómo corregirlo? (mayor transferencia)

Autorregulación: ¿qué reflexionar? (aprendizaje profundo)

SISTEMA (SLM o reglas):

Formato: TAREA / PROCESO / REFLEXIÓN.

Máx. 120 palabras. Sin código fuente.

Incluye nivel rúbrica + dimensiones + SHAP.

Filtro anti-repetición que garantiza feedback válido; si el modelo falla, las reglas cubren el mismo formato

Ejemplo de feedback generado:

TAREA:

Tu código obtiene **6/100** (Suspense, SS). Los factores más críticos según SHAP son la familia F (imports sin usar, SHAP = +2,1) y la ausencia de docstrings (SHAP = +1,7).

PROCESO:

1. Ejecuta `ruff check --fix` para eliminar imports muertos automáticamente.
2. Añade docstrings a cada función explicando qué hace, qué recibe y qué devuelve.
3. Divide `process_data` (CC=15) en sub-funciones.

REFLEXIÓN:

¿Cómo decides qué merece una función propia?
¿Qué criterio usas para nombrar variables?

Comparativa: feedback genérico vs. feedback seLIA2



ChatGPT / corrector genérico

"Tu código tiene algunos problemas.

Hay imports sin usar y la función

process_data es un poco larga.

Te sugiero revisar las buenas

prácticas de Python. . ."

- El alumno sabe que algo falla,
- pero no qué corregir primero.
- No hay nota ni criterio explícito.
- No hay estrategia concreta.

seLIA2 (TAREA/PROCESO/REFLEXIÓN)

"TAREA: 6/100 (E). Factor crítico: fam. F

(SHAP = +2,1).

PROCESO: ruff --fix + docstrings + dividir.

REFLEXIÓN: ¿cómo decides qué merece una función propia?"

- Tres niveles con mayor transferencia.
- Nota + criterio explícito.
- Estrategias concretas por dimensión.
- Preguntas de autorregulación.

Estación 5 — Informes Markdown: alumno y cohorte



Informe individual

Un .md por estudiante con:

- Tabla de dimensiones (corrección, estilo, complejidad, documentación)
- Justificación de cada penalización
- Feedback en 3 niveles
- Factores SHAP principales

Listo para subir al LMS (Moodle, etc.)

Informe de cohorte

cohorte.md con visión global:

- Distribución de niveles (SS-SB)
- Nota media de la clase
- Top 10 defectos frecuentes (qué re-enseñar)
- Medias por dimensión con barras
- Diagnósticos SHAP (fidelidad, estabilidad, ranking global)
- Ranking completo por nota

El docente ve **qué re-enseñar** a toda la clase; el alumno ve **qué mejorar** en su entrega.

Arquitectura: paquete instalable selia2/



Estructura modular

cli.py	CLI: analyze / datagen
config.py	Umbrales y pesos (auditable)
models.py	Dataclasses tipadas
features.py	50-feature vector (verdad)
scoring.py	Rúbrica 4D + niveles RD 1125/2003
analysis/	Ruff, Radon, AST runners
explain/	Surrogate + SHAP + diag.
feedback/	Reglas, prompts, SLM client
report/	Markdown (individual + cohorte)
datagen/	Generador paramétrico

Comandos principales

```
# Generar dataset
python -m selia2 datagen data/ --n 120

# Analizar con rúbrica + SHAP + informes
python -m selia2 analyze entregas/
    --explain --report out

# Sin SLM (solo reglas)
python -m selia2 analyze data/
    --explain --no-slm

# Ablaciones
python -m selia2 analyze data/
    --explain --no-slm --no-metric
python -m selia2 analyze data/
    --explain --no-slm --no-ast
```

Servidor SLM opcional (Ollama :11434 o propio :8765).
Sin servidor → feedback por reglas (misma calidad estructural).

Innovación y contribuciones de seLIA2

1 Rúbrica criterial auditable

Nota 0–100 (=0–10) según RD 1125/2003 (SS/AP/NT/SB); cada penalización se justifica y todos los umbrales son públicos en config.py.

2 Explicabilidad verificada

SHAP con fidelidad (CV accuracy) y estabilidad (Spearman entre semillas); si no son fiables, lo advierte al docente.

3 Feedback Hattie & Timperley

cd Tres niveles: tarea (qué falla), proceso (cómo corregirlo) y autorregulación (qué reflexionar).

4 50 features estructurales

Ruff (lint), Radon (complejidad) y AST (docstrings, type hints, anidamiento, bare excepts, nombres cortos...) en un vector único y auditable.

5 100 % local y privado

El código del alumno nunca sale del aula: 0€ por entrega, sin terceros, RGPD, 4 GB RAM. Independiente del LLM.

6 Informes + dataset reproducible

Markdown por alumno + cohorte (LMS-ready). Generador paramétrico: 120+ archivos, 24 perfiles, semilla fija.

Fundamentos pedagógicos

- Hattie, J., & Timperley, H. (2007). The power of feedback. *Rev. Educ. Res.*, 77(1), 81–112.
- Sadler, D. R. (1989). Formative assessment and the design of instructional systems. *Instr. Sci.*, 18, 119–144.
- Nicol, D., & Macfarlane-Dick, D. (2006). Formative assessment and self-regulated learning. *Stud. High. Educ.*, 31(2), 199–218.
- Black, P., & Wiliam, D. (1998). Assessment and classroom learning. *Assess. Educ.*, 5(1), 7–74.
- Shute, V. (2008). Focus on formative feedback. *Rev. Educ. Res.*, 78(1), 153–189.
- Keuning, H., Jeuring, J., & Heeren, B. (2018). A systematic review of automated feedback for programming exercises. *ACM TOCE*, 19(1).
- Messer, M., Brown, N. C. C., Kölling, M., & Shi, M. (2024). Automated grading and feedback tools for programming education. *ACM TOCE*.

Métodos y herramientas

- Lundberg, S., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *NeurIPS 30*. (SHAP)
- Molnar, C. (2020). *Interpretable Machine Learning*. Cap. Global Surrogate. AA~IL~&~CC.
- McCabe, T. (1976). A complexity measure. *IEEE TSE*, SE-2(4), 308–320.
- ISO/IEC 25010:2023. Systems and software engineering — SQuaRE — System and software quality models.
- RD 1125/2003. Sistema europeo de créditos y sistema de calificaciones universitarias. *BOE*, 224.
- Barberá, I. (EDPB, 2025). AI Privacy Risks & Mitigations — LLMs.
- Ruff (MIT) · Radon (MIT) · AST (PSF)
- SHAP (MIT) · scikit-learn (BSD-3)
- Qwen 2.5 (Apache 2.0) · llama.cpp (MIT)

Código y pipeline: github.com/rharagon/seLIA2 (MIT).

Demostración del pipeline

Salida **real** capturada en ejecución · 100 % local · sin cambiar de portátil

```
PS C:\seLIA> python -m selia2 datagen data/synthetic --n 120 --seed 42  
[OK] 120 archivos generados (semilla 42)
```

```
PS C:\seLIA> python -m selia2 analyze data/synthetic --explain --report out  
[OK] rúbrica + SHAP + 120 informes + cohorte.md
```

```
PS C:\seLIA> python -m selia2 analyze data/examples/student_bad.py  
--explain  
[OK] análisis + feedback en 3 niveles
```

Las siguientes diapositivas reproducen la salida literal de estos comandos.

Demo - ACTO 1 - Un archivo problemático: student_bad.py

```
=====
seLIA2 - 1 archivo(s) | solo reglas
=====
[1/3] Analizando (Ruff + Radon + AST) y aplicando rubrica...
      SS 3.8/10 student_bad.py 10 issues CC= 3.6 MI= 54 doc=90%
[2/3] SHAP omitido (usa --explain para activarlo).
[3/3] Generando feedback (reglas)...

-----
student_bad.py - Suspenso (SS) 3.8/10 · 38/100 [reglas]
-----

## Tarea - que corregir
- L6: [F401] `os` imported but unused
- L7: [F401] `sys` imported but unused
- L8: [F401] `json` imported but unused
- L9: [F401] `math` imported but unused
- L28: [C901] `process_data` is too complex (15 > 10)
- L28: [PLR0912] Too many branches (19 > 12)
- L31: [F841] Local variable `temp_storage` is assigned to but never used
- L118: [F841] Local variable `unused_threshold` is assigned to but never used
- L130: [E501] Line too long (119 > 88)
- L131: [E501] Line too long (174 > 88)
- [complejidad] anidamiento 6 > 4 (-6 pts)
- [complejidad] MI 54 < 65 (-3 pts)
- [documentacion] cobertura de type hints 0% (-6 pts)
- [documentacion] 8 numeros magicos (-2 pts)

## Proceso - como abordarlo
1. Recorre la cabecera de imports y elimina los que no se usan; tu editor puede marcarlos en gris.
2. La funcion tiene demasiadas ramas. Extrae bloques con nombre propio: cada funcion, una responsabilidad.
3. La funcion es demasiado grande (parametros/ramas/sentencias). Agrupa parametros o divide en pasos.
4. Busca variables asignadas que nunca se leen: o faltaba usarlas (posible bug) o sobran.
5. Divide las lineas largas (>88 col.) extrayendo variables intermedias con nombre significativo.
6. Escribe primero el docstring de una linea ('que hace y que devuelve') y anade type hints a los parametros: te obligar a a clarificar el diseno.

## Autorregulacion - para reflexionar
- Antes de entregar: ¿has ejecutado el archivo y comprobado que no lanza errores con un caso de prueba?
- Si vuelves a este codigo dentro de un mes, ¿sabrias que hace cada funcion sin releerla entera?
```

Demo - ACTO 1 - Mismo archivo, dos motores de feedback

```
=====
seLIA2 - 1 archivo(s) | qwen2.5-1.5b-instruct-q4_k_m.gguf (llama.cpp directo)
=====
[1/3] Analizando (Ruff + Radon + AST) y aplicando rubrica...
      SS 3.8/10 student_bad.py 10 issues CC= 3.6 MI= 54 doc=90%
[2/3] SHAP omitido (usa --explain para activarlo).
[3/3] Generando feedback (IA local)...
      student_bad.py... 24.2s

-----
student_bad.py - Suspenso (SS) 3.8/10 · 38/100 [slm]
-----

### TAREA: Corrección de los errores mencionados en la evaluación

### PROCESO:
1. Eliminar la importación inútil de `os`, `sys`, `json` y `math`
2. Mejorar la complejidad del método `process_data` reduciendo los bloques if a 10
3. Descomentar los bloques if que no están siendo utilizados
4. Normalizar el código para que todas las líneas sean cortas
5. Reemplazar cualquier variable que sea asignada pero nunca utilizada con `pass`

### REFLEXION: ¿Qué punto a mejorar en tu código en función de los cambios mencionados?
```

Demo - ACTO 1 - Código limpio con reglas: student_good.py

```
=====
seLIA2 - 1 archivo(s) | solo reglas
=====
[1/3] Analizando (Ruff + Radon + AST) y aplicando rubrica...
      NT 8.7/10 student_good.py          0 issues  CC= 2.6  MI= 59  doc=67%
[2/3] SHAP omitido (usa --explain para activarlo).
[3/3] Generando feedback (reglas)...

-----
student_good.py - Notable (NT) 8.7/10 · 87/100 [reglas]
-----

## Tarea - que corregir
- Sin issues de lint. Revisa abajo las dimensiones con menos puntos.
- [complejidad] MI 59 < 65 (-3 pts)
- [documentacion] 5 numeros magicos (-2 pts)

## Proceso - como abordarlo
1. Escribe primero el docstring de una linea ('que hace y que devuelve') y anade type hints a los parametros: te obligar
a a clarificar el diseno.

## Autorregulacion - para reflexionar
- Si vuelves a este codigo dentro de un mes, ¿sabrias que hace cada funcion sin releerla entera?
- ¿Puedes explicar cada funcion en una sola frase? Si necesitas 'y ademas...', probablemente haya que dividirla.
```

Demo - ACTO 1 - Código limpio también recibe feedback: student_good.py

```
=====
seLIA2 - 1 archivo(s) | qwen2.5-1.5b-instruct-q4_k_m.gguf (llama.cpp directo)
=====
[1/3] Analizando (Ruff + Radon + AST) y aplicando rubrica...
      NT 8.7/10 student_good.py          0 issues  CC= 2.6  MI= 59  doc=67%
[2/3] SHAP omitido (usa --explain para activarlo).
[3/3] Generando feedback (IA local)...
      student_good.py... 21.3s

-----
student_good.py - Notable (NT) 8.7/10 · 87/100 [slm]
-----

### TAREA: corregir el uso de "pass" en lugar de "return None"

### PROCESO: reemplazar todas las ocurrencias de "pass" con "return None" en el bloque principal de "student_good.py". Verificar que el código continúe funcionando correctamente después de la corrección.

### REFLEXION: ¿Por qué es importante evitar la uso de "pass" en lugar de "return None"?
```

Demo - ACTO 2 - Del archivo al aula: 120 entregas + SHAP

```
PS C:\seLIA> python -m selia2 analyze data/synthetic --explain --no-slm --report out
===== seLIA2 -- 120 archivos | SHAP + solo reglas =====
[1/3] Analizando (Ruff + Radon + AST) y aplicando rúbrica...
  SS 2.3/10~ gen_kitchen_sink_001.py~~~9 issues~ CC=7.0~ MI=35~ doc=0%
  SB 9.6/10~ gen_clean_small_001.py~~~~0 issues~ CC=2.0~ MI=75~ doc=100%
  ... (118 archivos más)

[2/3] Entrenando surrogate (etiqueta: rúbrica  $\geq 50$ ) y calculando SHAP...
fidelidad CV=1.00~~estabilidad Spearman=0.94~~clases=(22, 98)~~features=50
Top 5 features (|SHAP| medio):
  loc~~~~~0.0545~~
  sloc~~~~~0.0536~~
  mi~~~~~0.0484~~
  ruff_distinct_codes~0.0394~~
  lloc~~~~~0.0320~~

[3/3] Generando feedback (reglas)...~~[OK] 121 informes + cohorte.md
```

Fidelidad (CV=1.00) y estabilidad (Spearman=0.94) garantizan que la explicación es fiable. Insight de grupo: no “este alumno tiene F841”, sino “dónde concentrar la clase”.

Distribución de niveles (media 7.4/10)

Nivel	Entregas	%
SB Sobresaliente	25	21 %
NT Notable	57	48 %
AP Aprobado	16	13 %
SS Suspenso	22	18 %

Medias por dimensión

corrección~~~32.2/40~

estilo~~~~~17.8/20~

complejidad~~15.3/20~

documentación~8.9/20~~~~~~

Defectos más frecuentes (qué re-enseñar)

Código	Descripción	Aps.
F401	import sin usar	49
F841	variable sin usar	32
E722	bare except	24
E711	comparación a None	21
E712	comparación a True	21
F811	redefinición	13
C901	función muy compleja	10

SHAP global (top): loc, sloc, mi, ruff_distinct_codes, ruff_F841.

Fidelidad 1.00 · Estabilidad 0.94 · 50 features.

El docente ve qué re-enseñar a toda la clase; cada alumno recibe su .md individual listo para el LMS. Todo generado *offline*.

gen_kitchen_sink_001.md

Calificación: Suspenso (SS) — 2.3/10 · 23/100

Dimensión	Pts	Máx
corrección	8	40
estilo	11	20
complejidad	4	20
documentación	0	20

Justificación auditable de la nota

corrección (−8 ×4): F401 re/os/time; F841 sin_uso

estilo (−6): 3 issues PEP 8; (−3): 70 % nombres de 1 letra

complejidad (−4): CC 7.0>5; (−6): anidamiento 21>4;
(−6): MI 35<50

documentación (−10): docstrings 0 %; (−6): type hints 0 %

Prioridad: reducir tamaño/densidad —dividir en funciones

más pequeñas— es lo que más penaliza la nota.

Cada penalización queda registrada y justificada; los umbrales son públicos en `config.py`. Nota reproducible y defendible.

Demo - Y si hay pocas entregas?

```
=== 5. seLIA2 - batch + SHAP + reglas ===
6 ejemplos (data/samples): SHAP global, fidelidad y estabilidad. Set pequeno a proposito, para explicar los diagnostic
os.

=====
seLIA2 - 6 archivo(s) | SHAP + solo reglas
=====

[1/3] Analizando (Ruff + Radon + AST) y aplicando rubrica...
SB 9.0/10 sample_clean_models.py      0 issues CC= 2.2 MI= 46 doc=100%
SB 9.0/10 sample_clean_utils.py      0 issues CC= 2.3 MI= 48 doc=100%
AP 5.4/10 sample_comparisons.py     18 issues CC= 7.0 MI= 54 doc=0%
AP 5.8/10 sample_complex_logic.py    3 issues CC=10.0 MI= 43 doc=0%
AP 5.8/10 sample_style.py           16 issues CC= 7.5 MI= 65 doc=67%
SS 4.1/10 sample_unused_imports.py  15 issues CC= 1.3 MI= 68 doc=0%

[2/3] Entrenando surrogate (etiqueta: rubrica >= 50) y calculando SHAP...
fidelidad CV=n/d  estabilidad Spearman=0.71  clases=(1, 5)  features=50
[!] Fidelidad no calculable (clase minoritaria < 2): datos insuficientes para validar el surrogate.
[!] Estabilidad Spearman < 0.75: el ranking de features varia entre semillas; interpreta el orden con cautela.
Top 5 features (|SHAP| medio):
    ruff_fam_E          0.0111 #####
    ruff_sev_error      0.0098 #####
    mi                  0.0074 #####
    logical_density     0.0065 #####
    ast_avg_func_length 0.0064 #####

[3/3] Generando feedback (reglas)...
```

Demo - ACTO 3 - Pipeline completo con SLM local (Qwen 1.5B)

Feedback IA local, archivo a archivo

```
PS C:\seLIA> python -m selia2 analyze data/synthetic
--explain --report out

==== 28 archivos | SHAP +
qwen2.5-1.5b-instruct-q4_k_m.gguf
~~~~~(llama.cpp directo) =====

[3/3] Generando feedback (IA local)...
synth_almost_clean.py... ~~~~9.7s
synth_dead_code.py... ~~~~~20.5s
synth_high_cc_clean.py... ~12.6s
synth_long_function.py... (reglas: parroteó)
synth_mixed_heavy.py... ~~~16.0s
synth_ruff_driven_a.py... ~15.9s
synth_ruff_driven_c.py... (reglas: parroteó)
synth_undefined_name.py... ~7.5s
synth_volcano_cc.py... ~~~~~6.7s
... (28 archivos, 7-20 s cada uno)

[OK] 29 informes + cohorte.md
```

Lectura docente generada por el SLM

"Las calificaciones de la clase están influenciadas principalmente por los errores de pyflakes, las variables que no se usan, la concentración de errores y la densidad del archivo. Es crucial reforzar la mantenibilidad del código y reducir la densidad de los archivos."

Diagnósticos SHAP (mismo batch)

fidelidad CV=0.83~~estabilidad=0.93~~clases=(6,22)

ruff_fam_F	~~~~~0.0681~	
ruff_F841	~~~~~0.0357~	
density_issues_per_loc	~0.0246~	
ruff_sev_error	~~~~~0.0243~	
logical_density	~~~~~0.0171~	

Inferencia 100 % local (~7-20 s/archivo, sin red ni API). El filtro anti-repetición detecta cuando el modelo "parrotea" y cae a reglas sin perder el formato de 3 niveles.

Demo - ACTO 3 - Panel de aula: cohorte_slm.md vs cohorte.md

Distribución (28 entregas · media 6.4/10)

Nivel	Entregas	%
SB Sobresaliente	0	0 %
NT Notable	10	36 %
AP Aprobado	12	43 %
SS Suspenso	6	21 %

Medias por dimensión

corrección~~~27.8/40~

estilo~~~~~16.1/20~

complejidad~~14.8/20~

documentación~5.0/20~ SHAP: fidelidad 0.83 · estabilidad

0.93 · top ruff_fam_F, ruff_F841, density_issues_per_loc.

Defectos más frecuentes (qué re-enseñar)

Código	Descripción	Aps.
E501	línea demasiado larga	26
F841	variable sin usar	22
F401	import sin usar	12
E711	comparación a None	12
E722	bare except	7
C901	función muy compleja	7
PLR0912	demasiadas ramas	7

cohorte.md y cohorte_slm.md comparten estas tablas deterministas. La versión _slm añade además la “Lectura docente (IA local)” (diapositiva anterior): el mismo dato, resumido en lenguaje natural para el profesor.

Un solo comando produce el panel de aula y un .md por alumno. La capa de reglas garantiza las tablas; la capa de IA añade la narrativa —nunca al revés.

Demo - ACTO 3 - Ejemplo de ejecución

```
=====
seLIA2 - 28 archivo(s) | SHAP + qwen2.5-1.5b-instruct-q4_k_m.gguf (llama.cpp directo)
=====

[1/3] Analizando (Ruff + Radon + AST) y aplicando rubrica...
NT 7.2/10 synth_almost_clean.py          5 issues CC= 3.5 MI= 62 doc=80%
AP 6.0/10 synth_bare_except.py           6 issues CC= 2.2 MI= 73 doc=83%
AP 7.0/10 synth_borderline_cc.py         3 issues CC=11.0 MI= 59 doc=67%
NT 8.6/10 synth_borderline_mi.py         0 issues CC= 3.8 MI= 68 doc=80%
NT 8.3/10 synth_clean_deterministic.py   3 issues CC= 4.8 MI= 52 doc=86%
NT 8.0/10 synth_clean_large.py           0 issues CC= 4.5 MI= 54 doc=86%
NT 8.6/10 synth_clean_medium.py          0 issues CC= 2.0 MI= 71 doc=83%
NT 8.2/10 synth_clean_small.py           0 issues CC= 1.2 MI= 89 doc=20%
SS 4.4/10 synth_dead_code.py             6 issues CC= 2.2 MI= 66 doc=80%
AP 6.8/10 synth_deeply_nested.py         3 issues CC=10.5 MI= 59 doc=67%
NT 7.7/10 synth_dense_math.py           3 issues CC= 2.2 MI= 58 doc=80%
AP 5.7/10 synth_duplicate_functions.py   3 issues CC= 2.1 MI= 71 doc=12%
AP 6.0/10 synth_global_abuse.py          3 issues CC= 1.2 MI= 76 doc=17%
AP 6.8/10 synth_high_cc_clean.py         4 issues CC=12.0 MI= 54 doc=67%
NT 7.0/10 synth_if_chain.py             7 issues CC=10.0 MI= 55 doc=75%
AP 6.3/10 synth_long_function.py         4 issues CC=17.0 MI= 48 doc=67%
NT 7.7/10 synth_low_mi_clean.py          2 issues CC= 5.5 MI= 53 doc=80%
AP 5.8/10 synth_medium_ruff_e.py         5 issues CC= 2.5 MI= 70 doc=20%
SS 3.0/10 synth_mixed_heavy.py           9 issues CC= 3.5 MI= 65 doc=20%
SS 3.9/10 synth_ruff_driven_a.py         7 issues CC= 2.7 MI= 73 doc=25%
SS 4.2/10 synth_ruff_driven_b.py         8 issues CC= 1.6 MI= 83 doc=17%
AP 5.9/10 synth_ruff_driven_c.py        10 issues CC= 3.6 MI= 63 doc=17%
SS 4.0/10 synth_ruff_driven_d.py         5 issues CC= 2.0 MI= 72 doc=20%
AP 6.6/10 synth_scattered_issues.py      8 issues CC= 1.4 MI= 78 doc=83%
AP 5.0/10 synth_trailing_spaces.py       6 issues CC= 1.7 MI= 74 doc=14%
SS 4.8/10 synth_undefined_name.py       4 issues CC= 1.3 MI= 83 doc=75%
NT 8.2/10 synth_varied_halstead.py       2 issues CC= 2.2 MI= 70 doc=80%
AP 6.6/10 synth_volcano_cc.py            3 issues CC=11.0 MI= 59 doc=67%

[2/3] Entrenando surrogate (etiqueta: rubrica >= 50) y calculando SHAP...
fidelidad CV=0.83 estabilidad Spearman=0.93 clases=(6, 22) features=50
Top 5 features (|SHAP| medio):
ruff_fam_F          0.0681 #####
ruff_F841           0.0357 #####
density_issues_per_loc 0.0246 #####
ruff_sev_error      0.0243 #####
logical_density     0.0171 #####

Lectura docente (IA local):
Las calificaciones de la clase están influenciadas principalmente por los errores de
pyflakes, las variables que no se usan, la concentración de errores en el código y
la densidad del archivo. Los errores de pyflakes que no definen los nombres o no
redefinen correctamente son más importantes que los errores graves en la familia F.
Además, es crucial reforzar la concentración en la mantenibilidad del código y
reducir la densidad de datos en los archivos.

[3/3] Generando feedback (IA local)...
synth_almost_clean.py... 10.4s
synth_bare_except.py... 12.2s
```

Demo - ACTO 3 - Ejemplo de informe de cohorte 1

Informe de cohorte — seLIA2

28 entregas analizadas el 2026-07-04.

Distribucion de niveles

Nivel	Entregas	%
SB (Sobresaliente)	0	0%
NT (Notable)	10	36%
AP (Aprobado)	12	43%
SS (Suspense)	6	21%

Puntuacion media: **6.4/10** (64/100)

Matricula de Honor (propuesta)

Ningun alumno alcanza 9,0; no procede.

Defectos mas frecuentes (que reenseñar)

Codigo	Mensaje tipico	Apariciones	Entregas afectadas
E501	Line too long (96 > 88)	26	12
F841	Local variable backup_transactions is assigned to but never	22	9

Demo - ACTO 3 - Ejemplo de informe de cohorte 2

Dimensiones (media de la cohorte)

- correccion: 27.8/40 #####
- estilo: 16.1/20 #####
- complejidad: 14.8/20 #####
- documentacion: 5.0/20 ##

Explicabilidad (surrogate SHAP)

- Muestras: 28 (6 suspensos / 22 aprobados), 50 features.
- Fidelidad (accuracy CV): 0.83
- Estabilidad (Spearman entre semillas): 0.93

Top 10 features por |SHAP| medio (con interpretacion)

- errores de pyflakes: nombres sin definir, o imports/variables sin usar o redefinidos — `ruff_fam_F` (0.0681)
#####
- variables a las que se asigna un valor que luego no se usa — `ruff_F841` (0.0357) #####
- concentracion alta de problemas en poco codigo — `density_issues_per_loc` (0.0246)
#####
- alta concentracion de errores graves (familia F de pyflakes) — `ruff_sev_error` (0.0243)
#####
- tamano o densidad elevada del archivo — `logical_density` (0.0171) #####
- alta concentracion de errores graves (familia F de pyflakes) — `ruff_total` (0.0147)
#####

Demo - ACTO 3 - Ejemplo de informe de cohorte y 3

Lectura docente (IA local)

Las calificaciones de la clase están influenciadas principalmente por los errores de pyflakes, las variables que no se usan, la concentración de errores en el código y la densidad del archivo. Los errores de pyflakes que no definen los nombres o no redefinen correctamente son más importantes que los errores graves en la familia F. Además, es crucial reforzar la concentración en la mantenibilidad del código y reducir la densidad de datos en los archivos.

Entregas (ordenadas por puntuacion)

Archivo	Nivel	Puntos	Issues	CC	MI
synth_clean_medium.py	NT	86	0	2.0	71
synth_borderline_mi.py	NT	86	0	3.8	68
synth_clean_deterministic.py	NT	83	3	4.8	52
synth_clean_small.py	NT	82	0	1.2	89
synth_varied_hastead.py	NT	82	2	2.2	70
synth_clean_large.py	NT	80	0	4.5	54
synth_dense_math.py	NT	77	3	2.2	58

DEMO

```
# Generar dataset
python -m selia2 datagen
data/synthetic --n 120

# Analizar con todo
python -m selia2 analyze
data/synthetic --explain --report out

# Ver informe de cohorte
cat out/cohorte.md
```

¿Preguntas?

Agradecimientos

Gregorio Robles · Jesús M. González Barahona
Dpto. Teoría de la Señal y Comunicaciones y Sistemas
Telemáticos y Computación — URJC

Organización de seLIA 2026

Herramientas que lo hacen posible:

Ruff (MIT) · Radon (MIT) · AST (PSF)
SHAP (MIT) · scikit-learn (BSD-3)

Qwen 2.5 (Apache 2.0) · llama.cpp (MIT)

github.com/rharagon/seLIA2

Código del pipeline: MIT